# 解题配方库

agent-recipe-book —— 让不能联网的 AI，也能积累经验

正文 8 页（含本封面）· 代码仓库与线上演示地址见正文首页

# 解题配方库 agent-recipe-book

**AI 不是不会，它只是不记得自己踩过坑。本库把踩过的坑存成机器可读的“配方”，挂进 AI 的工具箱——下次遇到同类问题它自己去查，不出网、不买 API、也无需换更贵的显卡。**

仓库 github.com/taonai4097-prog/agent-recipe-book · 线上浏览 taonai4097-prog.github.io/agent-recipe-book · 全量数据 api/experiences.json

---

> **一句话：**把 AI 想成一个**每天早上失忆的博士**——有能力，但不记得自己踩过什么坑，每天都从零重来；**而且就算能力再强，也总有踩到坑、绕远路的时候。**本库就是递给他一本**共享的错题本**：不一定让他更聪明，但让他**少绕几圈、早点收工。**它不替 AI 思考，只替 AI **不重复犯错。**

## 一、两个真实的困境

### 困境一：同一个坑，换个窗口又踩一遍

一个开发者让 AI 写采集脚本。第一次，AI 用 requests 抓中文站，**乱码。**它试了改编码、改 headers、换库、手动解 gzip……绕了四轮才跑通。这一轮烧掉的 token，够把这段代码写三遍。

两周后他开了个新项目，又碰到中文站乱码。AI **不记得上一次**，又从零开始绕——这次它挑了另一条路：一个早该放弃的解析库。脚本勉强能跑，但边界情况一堆 bug，**效果还不如第一次。**

他想过去换个更强的模型。但 a16z 的刷卡数据很扎心：**付费者里只有 13% 会买第二个 AI 产品**——绝大多数人停在一个套餐上。为一个已经解决过的乱码去升级，不值得，也懒得动。

### 困境二：数据出不了内网的医院

一家医院上了本地大模型。病历数据出不了内网，只能用 7B 这样的小模型；而医院那批电脑很多还是几年前的配置，显存吃紧，**再大的模型根本跑不动。**

用了三个月，科室反馈两件事：

- **能力跟不上**——同样的问题，云端大模型一遍就过，本地的要绕三遍
- **反复在同一个地方摔跤**——内网代理用的是自签证书、HIS 接口某个字段的含义要猜……上周刚解决过，这周换个对话窗口，它又从零绕一遍

第二件比第一件更磨人。因为它不是“慢”，是**白干。**

### 那常规办法走得通吗

| 想到的办法 | 为什么走不通 |
| --- | --- |
| 换个更贵的模型套餐 | **可以，但几乎没人这么做**——a16z 数据显示，付费者里仅 13% 会买第二个产品，多数人停在一个套餐上。而对医院还有一道硬门槛：**数据出不了内网**，再贵也用不上；就算只算账，采购审批按月计、另购服务器、7×24 电费运维，也不是一个科室能拍板的开支 |
| 换成更大的本地模型 | 医院那批电脑的配置跑不动；而且**内网特有的坑，模型再大也没见过——不是参数不够，是没见过** |
| 把经验写进科室 wiki | AI 是**反应式**的，它不会主动去翻一本外部册子；人也不会每次开工都先翻一遍 |
| 微调 | 要重训、要算力、不可逆；**改一条经验得重来一遍**，也没法逐条人工复核 |

所以真正的问题不是“模型不够强”，而是**经验没有载体**——经验散在一次次对话里，关掉窗口就没了。对 AI 来说，**每次开工，它都是一个新员工。**

**本节数据来源**：a16z《Top 100 消费级 AI 应用》第七版（2026-10-05），付费数据来自 YipitData 美国消费者刷卡**抽样**面板（仅覆盖美国）；中文解读见数字生命卡兹克。仅用于说明**外部环境**，不影响本库自身的实验口径。

## 二、我们的做法

把「踩坑过程」从对话里搬到文件里，再挂进 AI 的工具箱。**存在文件里，就不会像聊天记录一样被清空。**

| | 一般知识库 | 本库 |
| --- | --- | --- |
| 装什么 | 答案（“这样做就行”） | 试错过程（试了 X 失败因为 Y、试了 Z 也失败、最后 A 成了） |
| 拿去怎么用 | 照抄 | 自己判断“A 适不适合我” |
| 怎么进 AI 脑子 | 人去翻 | 挂成 MCP 工具，AI 遇事**自己检索**（只给最匹配的 1 条） |

每条配方里最看重的字段叫 `dead_ends`（死胡同），装的是**已经排除的选项**。知道“哪条路走不通”，往往比知道“正确答案”更省时间——库内 169 个死胡同，字段完整率 **100%**。

### 它能做到什么 理论上

- **少绕几圈**——同一个坑不再从零试起，省下的是轮次、token 和等待时间
- **少花算力**——本地小模型少一轮重试，就少一份显存占用、一份电
- **结果更稳**——不会第二次又拐进一条“效果更差、bug 更多”的岔路
- **把一个人的坑变成一批机器的坑**——院内一台机器踩过的坑，内网共享后全科室不再重踩，**数据一行都不出院区**
- **越用越值钱**——库是资产，写进去就一直在，不会随模型换代归零

**为什么必须挂成工具**：AI 是反应式的，它不会主动去翻一本外部册子；只有挂成 MCP 工具，它才“看得见”。

**坑从哪来**：通用坑（编码、git、环境、工具链这类与业务无关的）从公开库**联网取一次**即可，之后永久离线；院内的特异性坑由院内机器自己攒、走**院区内网**共享——数据不出院区。

**顺带说，这个方向被顶级风投点名过。**a16z 在榜单里给初创公司总结了四条生存经验，其中一条是**“钻进巨头不愿碰的隐秘角落——合规政策与隐私顾虑，恰恰是初创公司的护城河”**，并举了主打医疗隐私的 OpenEvidence（已渗透美国五到六成医生）为例。本库的主线正对着这个角落：**数据出不了内网的地方。**

## 三、和别的方案有什么不一样

| 对比对象 | 它解决什么 | 关键差别 |
| --- | --- | --- |
| **RAG** | 让模型知道没见过的资料 | RAG 给**事实与答案**；本库给**试错过程**（重点是哪条路走不通）。RAG 通常要 embedding + 向量库，本库纯文本、运行时零依赖 |
| **记忆类项目** | 记住用户是谁、偏好什么 | 它们存**人**，本库存**坑**；且多以服务形式提供，数据要落服务端 |
| **项目规则文件**（`.cursorrules` / `CLAUDE.md`） | 约束 AI 应该怎么做 | 规则是**要求**（全量注入、越加越长）；配方是**经历**（按需检索，只给 1 条）。实测给 1 条优于给 5 条（**+3.8pp**） |
| **提示词模板库** | 复用一段好用的提示词 | 模板是**一次性输入**，不记录失败；配方记录**失败路径**，且不进常驻上下文 |
| **微调** | 把能力烧进权重 | 改权重要重训、不可逆；本库改文件，**秒级更新、可回滚、可逐条复核** |

**一句话区别**：它们大多在解决“AI 不知道”或“AI 记不住”；本库解决的是**“AI 不知道队友已经踩过这个坑”**——而且是在**不能联网**的前提下。

## 四、工程与产品完成度

### 库里现在有什么

| | |
|---|---|
| 配方总数 | **66 条**（全部 published） |
| 其中 A 档（实测可复现） | **54 条** |
| 死胡同（已排除的选项） | **169 个** |
| 标签 | **96 个** |
| 贡献者 | **3 名**（匿名标识；含 2 位资深程序员评审，其反馈已用于项目调整） |

覆盖**网页采集 / 网站工程 / 项目判断 / 工具链**四个领域。**多租户**：同一份代码，靠两个环境变量换库——框架与数据分离。现已有两个私有租户示例（LifeLens 7 条 / 小红书 6 条），配置见 `docs/PRIVATE-DEPLOYMENT.md`。

### 做完了什么

| 层面 | 交付 |
|---|---|
| **AI 侧（读）** | 5 个 MCP 工具：`search_recipes` / `get_recipe` / `list_tags` / `list_quarantine` / `verify_citations` |
| **AI 侧（写）** | 2 个工具：`submit_recipe` 等。Agent 提交 → **隔离池** → 人复核才公开；四道留痕：脱敏复检双挂 / `status + confidence` 隔离双锁 / A 档 `verified` 门槛 / C→B→A 禁跳级 |
| **人侧** | 本地网站作为线上演示（浏览 / 检索 / 详情 / 标签 / 隔离池） |
| **部署** | 运行时**零第三方依赖**（跑测试无需 `npm install`）、零上传、不需要任何 API Key；**多租户**支持 3 个库同时挂载、互不污染 |
| **质量门禁** | **9 个测试文件、304 条断言全绿**；CI 双 job（行为 + 静态检查）每次推送跑全量 |
| **对抗测试** | 红队 31 发攻击用例，3 发擦伤**当天修复并固化为回归测试**（`tests/` 内 RT-1~RT-5） |
| **可复现** | 干净克隆 → 304/0；HTTP 起服务后网页翻页 66 条齐全 |

## 五、效果数据

同一个本地模型做同一批题，**一次不给配方、一次给。**

| 指标 | 数值 |
| --- | --- |
| 平均提升 | **+8.0 个百分点（pp）** |
| 巧合概率 | **p=0.0102**（另两种算法 0.0086 / 0.0262，都站得住） |
| 稳定性 | **留一法 42/42** 全部保持显著——不靠任何一道题撑 |
| 通过率 | **11.9% → 21.4%**（相对提升 80%） |

题库 42 道 × 四臂 × 双模型 = **336 条真实运行记录**（qwen2.5-coder:7b / qwen2.5:7b，全部本地运行、零上传）。四臂：① 什么都不给 ② 给检索到的第 1 条 ③ 给前 5 条 ④ 给**无关的**配方（安慰剂）。

两个额外发现：**① 给得越少越好**——只给第 1 条比给前 5 条显著更好（+3.8pp，p=0.0306），给一堆会稀释效果；**② 安慰剂组反而更差**（−4.8pp）——效果来自“给对了”，不是“多塞了文字”。这也解释了为什么我们只给 top-1：**给错配方不是没帮上忙，而是会造成主动伤害。**完整实验设计与统计口径见仓库内 `EVIDENCE.md`。

## 六、可优化的点，也是下一步

> 下面这几条不是免责声明，是**已经看清了的迭代方向**——每一条都对应一个具体的下一步。

| 现象 | 我们的判断与下一步 |
| --- | --- |
| 强模型组测不出增量（p=0.531） | 该组**对照组基线已经 9/10**，只剩 10% 抬升空间——这是**天花板效应**，不是配方无效。下一步：把题库换成模型**没见过的内网特异性坑**再测（此项尚未实测） |
| 常见坑上零提升（p=1.000） | 编码、cookie 这类，裸跑就是 100%——**说明这类配方不该进库。**下一步：做**配方价值分层**，把库容量留给“模型不会的坑” |
| 错配会造成主动伤害 | 已用“只给 top-1 + 人工复核”兜住。下一步：加**场景匹配度校验**——匹配不上时宁可一条都不给 |
| 外部使用者还是 0 人 | 投稿通道已开放（见 §十）。下一步：面向一批本地部署场景发出试用邀请，把“有没有人真的需要”补成外部证据 |
| 院区多机之间的库同步 | 本库提供换库机制，机器间同步需院区自建（内网 git / 共享盘 / 摆渡）。下一步：给出一个可一键部署的**院区内同步方案** |

另外三项仍在队列里：**跨模型泛化**（目前只在 qwen2.5-coder:7b / qwen2.5:7b 上测过）、**并发写入**（低危留档，未压测）、**长期效果**（库用久了会不会退化，需要时间跨度）。

> **顺带说一件我们做过的事**：审计实验时我们发现一个组的对照基线其实是空的（环境故障，不是模型答错），于是**主动撤回了自己先前的一个结论**，并把“环境失败要自动识别、单独统计、从分母剔除”写进了测试工具。现在这套自检是回归测试的一部分。

## 七、三步用起来

```
# 1. 拿到仓库（公开，无需密钥）
git clone https://github.com/taonai4097-prog/agent-recipe-book.git
cd agent-recipe-book

# 2. 挂到你的 AI 客户端（MCP 配置里加一段，见 mcp/README.md）
#    command: node   args: ["<绝对路径>/agent-recipe-book/mcp/server.js"]
#    需要 Node 18+，运行时零第三方依赖、零上传、不需要任何 API Key

# 3. 起本地网页预览（必须走 http 协议，双击 html 不行）
python -m http.server 8099     # 打开 http://127.0.0.1:8099/
```

## 八、复现全部数字

```
# 库规模（应输出 66）
curl https://taonai4097-prog.github.io/agent-recipe-book/api/experiences.json \
  | python -c "import sys,json;print(len(json.load(sys.stdin)))"

# 全量测试（口径以末行「总计」为准，当前 304 通过 / 0 失败）
npm test

# 重建索引（用带 pyyaml 的解释器）
python api/ingest.py rebuild

# 一次真实检索（MCP 直连，无需客户端）
node mcp/server.js < mcp/demo-requests.jsonl
```

最后一条期望看到：`已加载 66 条配方` → `"matched": 19`，首条 `recipe-py-encoding-mojibake`（匹配度 100%、置信 A）。请求体就是仓库里的 `mcp/demo-requests.jsonl`，三行 JSON-RPC，可直接改。

## 九、开源许可

- **代码**（网站 / MCP / 检索内核 / 脚本）：**AGPL-3.0** © 2026 Laoyang4097。自托管、自用、修改均可；若将修改版作为网络服务对外提供，须依 AGPL 开放完整修改源码。
- **配方数据**（`recipes/*.md` 及 `api/experiences.json` 的内容）：**CC BY-SA 4.0**。投稿即同意随库发布，并授予维护者不受“相同方式共享”约束的商用使用权（详见 `CONTRIBUTING.md`）。

## 十、它想长成什么样

把场景放大一点：**一排工位，每台机器上坐着一个“每天失忆的博士”**——都很聪明，却每天从零开始，谁踩过的坑都不会告诉下一个。这个库想做的事就一句话：**给他们一本共享的错题本。**

这件事不只我们在说。头部 AI 博主卡兹克总结自己用法时讲过三句话：**“用了不记录，等于没用”“出了错不是改了就结束了，是追问为什么，然后写成规则”“把调教过程沉淀成资产”**——这几乎就是本库 `dead_ends` 字段的民间版。他还说过一句更直白的：**“并不是所有人，都有条件去买 ChatGPT 的会员的。”**

现在这本错题本只有 **66 条、来自 3 个人。“框架对不对”和“库够不够好”是两件事**：框架站得住，库还太小，真正稀缺的是**配方的质量与数量。**

**所以最缺的是配方，尤其是那种“只有你们那儿才有”的坑**——外面搜不到、书上写不进、只能吃过亏才知道。提交一条配方 = 提一个 PR，无需账号系统，进隔离池、经人复核后公开（规则见 `CONTRIBUTING.md`）。**让每一个踩过坑的人，都为后来者省下那一步。**

---

解题配方库 agent-recipe-book · 2026-10-05 定稿